Chapter 1: Backpropagation and Gradient Flow

Backpropagation is the algorithm that makes training deep neural networks practical. It computes the gradient of a loss function with respect to every weight in the network by applying the chain rule of calculus repeatedly from the output layer back to the input layer. Without backpropagation, computing gradients for a network with millions of parameters would require a separate forward pass for each parameter, making training computationally infeasible.

1.1 The Chain Rule in Neural Networks

The chain rule states that if y = f(g(x)), then dy/dx = (dy/dg) * (dg/dx). In a neural network with layers L1, L2, L3, the gradient of the loss with respect to weights in L1 requires multiplying the local gradients at each layer. This is why deep networks can suffer from the vanishing gradient problem — multiplying many small numbers together produces a result that approaches zero, making early layers learn extremely slowly or not at all.

1.2 Vanishing and Exploding Gradients

The vanishing gradient problem occurs primarily with sigmoid and tanh activations, whose derivatives saturate near zero for large input values. When gradients shrink through many layers, weights in early layers receive almost no update signal. The exploding gradient problem is the opposite — gradients grow uncontrollably, causing weight updates that destabilize training. Gradient clipping is a common solution for exploding gradients: if the gradient norm exceeds a threshold, it is scaled down to that threshold before the update is applied.

1.3 Solutions: Residual Connections

Residual connections, introduced in ResNet, address vanishing gradients by adding a skip connection that allows gradients to flow directly from later layers to earlier ones without passing through the intermediate transformations. The residual block computes F(x) + x instead of just F(x), where x is the identity shortcut. During backpropagation, the gradient of the loss with respect to x receives a direct path (gradient of 1 from the identity) plus the gradient through F(x). This simple addition enabled training of networks with over 100 layers.

Chapter 2: Optimization Algorithms

Choosing the right optimizer significantly affects both training speed and final model quality. All gradient-based optimizers share the same core idea: use the gradient to determine which direction to move in parameter space, and how far. They differ in how they adapt the learning rate and how they use information from previous gradients.

2.1 Stochastic Gradient Descent

Vanilla SGD updates weights by subtracting the gradient multiplied by a fixed learning rate: w = w - lr * gradient. The term stochastic refers to using a random mini-batch of data rather than the full dataset for each update. This introduces noise into the gradient estimate, which can actually help escape sharp local minima. However, SGD is sensitive to the choice of learning rate and can oscillate in ravines — regions where the loss surface curves more steeply in one direction than another.

2.2 Momentum

Momentum accelerates SGD by accumulating a velocity vector in the direction of consistent gradients. The update becomes: velocity = beta * velocity + gradient, then w = w - lr * velocity. The beta parameter (typically 0.9) controls how much previous gradients influence the current step. Momentum helps dampen oscillations in the ravine direction while accelerating in the consistent direction, leading to faster convergence. Nesterov momentum is a variant that computes the gradient at the anticipated future position rather than the current position, giving a more accurate correction.

2.3 Adam Optimizer

Adam (Adaptive Moment Estimation) combines momentum with adaptive per-parameter learning rates. It maintains two moving averages: the first moment (mean of gradients, like momentum) and the second moment (mean of squared gradients, like RMSProp). The learning rate for each parameter is scaled by the inverse square root of the second moment, causing parameters with large gradients to take smaller steps and parameters with small gradients to take larger steps. Adam includes bias correction terms for both moments because they are initialized at zero and are biased toward zero in early training steps. The default hyperparameters — lr=0.001, beta1=0.9, beta2=0.999, epsilon=1e-8 — work well across a wide range of problems without tuning.

2.4 Learning Rate Scheduling

A fixed learning rate is rarely optimal throughout training. Learning rate scheduling adjusts the learning rate during training according to a predefined policy. Step decay reduces the learning rate by a factor (e.g., 0.1) every fixed number of epochs. Cosine annealing smoothly decreases the learning rate following a cosine curve, sometimes with warm restarts (cosine annealing with restarts, or SGDR). Warmup schedules start with a very small learning rate and gradually increase it during the first few thousand steps — this is critical for training transformers, where jumping to a large learning rate from the start destabilizes the attention mechanism.

Chapter 3: Normalization Techniques

Normalization layers stabilize training by controlling the distribution of activations throughout the network. Without normalization, the distribution of inputs to each layer shifts as the parameters of previous layers change — a phenomenon called internal covariate shift. Different normalization techniques make different assumptions about the data and are suited to different architectures.

3.1 Batch Normalization

Batch Normalization (BatchNorm) normalizes activations across the batch dimension. For each feature, it subtracts the batch mean and divides by the batch standard deviation, then applies learned scale (gamma) and shift (beta) parameters. BatchNorm dramatically accelerates training and allows the use of higher learning rates. It also acts as a regularizer, reducing the need for dropout in some architectures. However, BatchNorm has two key limitations: it behaves differently during training (using batch statistics) versus inference (using running statistics collected during training), and it is ineffective for very small batch sizes or for recurrent networks.

3.2 Layer Normalization

Layer Normalization (LayerNorm) normalizes across the feature dimension instead of the batch dimension. For each sample independently, it computes the mean and standard deviation across all features in a layer, then normalizes. This makes LayerNorm independent of batch size, which is why it replaced BatchNorm in transformer architectures. In BERT, GPT, and other large language models, LayerNorm is applied either before the attention and feedforward sublayers (pre-norm, which stabilizes training of very deep networks) or after them (post-norm, the original transformer formulation).

3.3 Dropout as Regularization

Dropout is a regularization technique where random neurons are set to zero during training with probability p (typically 0.1 to 0.5). This prevents neurons from co-adapting, forcing the network to learn redundant representations. At inference time, dropout is disabled and activations are scaled by (1-p) to maintain the same expected value. In transformers, dropout is applied after attention weights and after each sublayer. Dropout rates in modern large language models are typically low (0.1) because the models are large enough that regularization from the data is dominant.

Chapter 4: Embeddings and Representations

Embeddings are dense vector representations of discrete objects — words, tokens, users, items — in a continuous vector space. The key property of a good embedding is that geometric relationships in the vector space correspond to semantic relationships in the original domain. Word2Vec demonstrated that vector arithmetic could capture analogies: the vector for king minus man plus woman approximates the vector for queen.

4.1 Word Embeddings

Word2Vec trains embeddings using two objectives: CBOW (predict the center word from context) and Skip-gram (predict context words from the center word). Skip-gram with negative sampling is more commonly used because it scales better to large vocabularies. GloVe (Global Vectors) takes a different approach, factorizing the word co-occurrence matrix directly. Both methods produce static embeddings — each word has one vector regardless of context — which means they cannot handle polysemy (words with multiple meanings depending on context).

4.2 Contextual Embeddings

ELMo (Embeddings from Language Models) introduced contextual embeddings by using a bidirectional LSTM to produce representations that depend on the full sentence context. BERT took this further with the transformer architecture and masked language modeling, producing representations that capture rich contextual information. In BERT, the embedding for the word bank in a financial sentence differs from the embedding for bank in a river context, solving the polysemy problem. These contextual embeddings form the backbone of modern NLP systems.

4.3 Positional Encodings

Transformers process all tokens simultaneously rather than sequentially, so they need explicit positional information. Positional encodings are vectors added to the token embeddings to inject position information. The original transformer used sinusoidal positional encodings, where each dimension uses a sine or cosine function of different frequency. This allows the model to attend to relative positions: for any offset k, PE(pos+k) can be expressed as a linear function of PE(pos). Learned positional embeddings (used in BERT and GPT) are simply an embedding table indexed by position, which the model learns during training.

Chapter 5: Attention Mechanisms in Depth

Attention allows a model to focus on relevant parts of the input when producing each output. Before attention, sequence-to-sequence models compressed the entire input into a single fixed-size context vector, creating an information bottleneck. Attention mechanisms resolve this by allowing the decoder to look back at all encoder states and compute a weighted combination, where the weights reflect relevance.

5.1 Scaled Dot-Product Attention

The attention function takes queries Q, keys K, and values V as input. The output is a weighted sum of the values, where the weight for each value is determined by the dot product of the query with the corresponding key. The dot products are scaled by the square root of the key dimension (dk) to prevent

the softmax from entering a region of extremely small gradients when dk is large. The formula is: Attention(Q,K,V) = softmax(Q*K^T / sqrt(dk)) * V.

5.2 Why Scaling Matters

Without scaling, the dot products grow large in magnitude as dk increases. For large values, the softmax function has very small gradients because most probability mass concentrates on a single value. Dividing by sqrt(dk) keeps the dot products in a reasonable range. For example, if dk=64, we divide by 8. If dk=512, we divide by approximately 22.6. This is one of those small implementation details that has a large effect on training stability.

5.3 Cross-Attention vs Self-Attention

In self-attention, queries, keys, and values all come from the same sequence. Each position attends to all other positions in the same sequence, allowing the model to capture long-range dependencies within a sequence. In cross-attention (used in the decoder of encoder-decoder models), queries come from the decoder while keys and values come from the encoder output. This allows the decoder to selectively focus on relevant parts of the input sequence when generating each output token. GPT-style models use only self-attention, while models like the original transformer and T5 use both.

5.4 Causal Masking

In autoregressive language models, the model must not attend to future tokens when predicting the current token. Causal masking (also called the look-ahead mask) sets attention weights for future positions to negative infinity before the softmax, causing them to receive zero attention weight after the softmax. The mask is an upper triangular matrix of negative infinities. This allows GPT-style models to be trained efficiently on all positions in parallel while maintaining the autoregressive property at inference time.

Chapter 6: Modern Training Techniques

Training large neural networks requires careful management of compute, memory, and stability. Several techniques have become standard practice for training large models efficiently and reliably.

6.1 Mixed Precision Training

Mixed precision training uses 16-bit floating point (FP16 or BF16) for most computations while maintaining a master copy of weights in 32-bit (FP32) for the optimizer update. FP16 operations are 2-8x faster on modern GPUs and require half the memory, enabling larger batch sizes or larger models. The risk of FP16 is numerical underflow — gradients can become zero if they are too small to represent in FP16. Loss scaling addresses this by multiplying the loss by a large constant before backpropagation

and dividing the gradients by the same constant afterward. BF16 (bfloat16) has the same exponent range as FP32 but less precision, making it more numerically stable than FP16 for training.

6.2 Gradient Checkpointing

During backpropagation, intermediate activations from the forward pass must be stored to compute gradients. For deep networks, this can consume enormous memory. Gradient checkpointing trades compute for memory by recomputing activations during the backward pass rather than storing them all. Only a subset of activations (the checkpoints) are stored; others are recomputed when needed. This typically increases computation time by 30-40% but can reduce memory usage by a factor of sqrt(n) where n is the number of layers.

6.3 Knowledge Distillation

Knowledge distillation trains a smaller student model to mimic a larger teacher model. Instead of training on hard labels (one-hot vectors), the student trains on soft labels — the probability distributions output by the teacher. Soft labels contain more information than hard labels: they reveal which classes the teacher considers similar (a truck is more likely to be confused with a car than a bird). The distillation loss combines the standard cross-entropy loss with the teacher on hard labels and a KL divergence loss between student and teacher probability distributions. DistilBERT demonstrated that a student with 40% fewer parameters than BERT can retain 97% of BERT performance on most benchmarks.